

1 Learning Aim

My learning aim is to understand better the limitations, benefits, and detriments of a continuous-space reinforcement learning algorithm.

Reinforcement learning has the ability to improve performance beyond that of a classic algorithm since it has knowledge of a reward signal and is designed to exploit that signal for maximum reward.

An issue with existing robot controllers is that they may be suboptimal, may not handle dynamic environments well, or may not be sufficiently intelligent to complete complex tasks. Neural nets trained via reinforcement learning can handle all three of these deficiencies, if properly set up and trained.

The aim of the learning algorithm is to learn a continuous policy to control a differential drive robot to navigate toward a goal, given a dataset of experiences which it collects during exploration. The policy will be goal-conditioned.

2 Learning Algorithm

I'll be using Soft Actor-Critic (SAC) as my reinforcement learning algorithm. One reason is that it allows for continuous input and output spaces. Another reason is that I use SAC in my PhD research and this will give me an opportunity to understand the algorithm more deeply. This will require me to write my own neural net and RL library, which shouldn't be a problem as I've already completed that for this project.

SAC works similarly to Q-Learning, in that it has a critic, the Q network, which evaluates (s, a) inputs and outputs q-values. And it has an actor policy, the π network, which produces actions as a function of the input (s, g) , where s is the state and g is the goal.

SAC suffers from the curse of dimensionality, meaning that it takes exponentially more data to fully map the search space as the dimension of the search space increases. But since the action-space for this problem is only 2d, I expect SAC to work well enough to control a differential drive robot.

For now my policy doesn't have a loss term for exploration, as is the case in the standard SAC. Instead, initial exploration will naturally be caused by the actor before it converges. If this leads to a low success rate, then an explicit exploration loss term will be added.

3 Problem Formulation

The inputs to the algorithm will be the state, (x, y, θ) , along with a goal tuple, (x_g, y_g) . The outputs of the algorithm will be the control variables, (v, ω) .

There will be a sparse reward of 1.0 if the goal was reached and 0.0 otherwise. In addition, a small movement energy penalty will be added to regularize the actions, which prevents them from becoming too large in magnitude. The movement penalty is -0.01 if the robot moves to an empty cell, and -1 if the cell is occupied.

The neural net will be parameterized by an arbitrary number of parameters, and their values will be tuned via gradient descent, and back-propagation.

The map and obstacles from DS1 will be used to create an environment. The motion model used in previous assignments will be used to drive the robot through the environment. Over the course of many episodes, the robot will explore online and collect data to create a replay buffer (a.k.a. a dataset).

If this problem proves too simple to solve, then dynamic obstacles or randomized obstacle locations will be added. In that case, the state will be appended with the robot's observation of surrounding obstacles.

4 Evaluation Measures

Metrics such as success rate, time-to-goal, and energy usage will be recorded. Metrics to track how well learning is happening will also be recorded, including average q-values, average episodic cumulative reward, and critic loss.